Healthcare Assistant

Report di Sviluppo del Progetto

Assistente Virtuale per il Settore Sanitario

Questo report vuole descrivere il processo di progettazione e sviluppo di un **Healthcare Virtual Assistant**, realizzato come soluzione alla challenge tecnica proposta da Silicon Shoring Reply.

L'obiettivo dell'esercizio è stato quello di dimostrare competenze nella progettazione di sistemi software, con particolare attenzione all'esperienza utente e all'usabilità, all'efficienza dell'architettura e all'attenzione per il dominio di riferimento, cioè quello sanitario.

Il documento illustra le scelte progettuali adottate, i principali requisiti funzionali e non funzionali, nonché le tecnologie usate o implementate per la realizzazione del sistema. Viene inoltre presentata una panoramica delle criticità affrontate durante lo sviluppo e delle relative soluzioni adottate, con l'obiettivo di evidenziare il processo decisionale seguito e la capacità di tradurre un problema tecnico in una soluzione concreta e scalabile.

Il progetto è consultabile e fruibile su GitHub:
https://github.com/francescapoli98/healthcare_virtual_assistant.

Come supporto durante la fase di sviluppo, è stato impiegato Claude (Anthropic) per attività di consulenza tecnica, debugging e approfondimento di specifiche scelte implementative.

1. Struttura del Sistema

Il progetto è strutturato secondo un'architettura a tre livelli (three-tier), con una separazione netta tra frontend, backend e dati, che dialogano tra loro attraverso routes.

1.1 Stack

Layer	Tecnologia	Ruolo
Frontend	React.js	Interfaccia utente — chat, prenotazioni, storico
Backend	Flask (Python)	API REST, logica RAG, triage, orchestrazione AI
Database	MySQL + Vector DB	Dati relazionali + embeddings per RAG

1.2 Struttura delle directory

Il repository è organizzato secondo una struttura modulare e containerizzata, che rispecchia innanzitutto la distinzione tra backend e frontend, ma anche la composizione per Blueprints del backend Flask.
Nella tabella di seguito vengono descritte le directory e i file principali più nel dettaglio per una migliore comprensione della struttura.

Percorso	Contenuto e ruolo
app/	Package principale del backend Flask. Contiene tutta la logica applicativa.
app/rag/	Modulo RAG: gestione degli embeddings, interrogazione dell'indice FAISS, composizione del contesto per il modello LLM.
app/routes/	Blueprint Flask con tutti gli endpoint REST (chat, pazienti, medici, appuntamenti, autenticazione).
app/__init__.py	Application factory: inizializza Flask, registra i Blueprint, configura le estensioni.
app/config.py	Configurazione centralizzata: stringa di connessione MySQL, chiavi API, parametri RAG e variabili d'ambiente.
app/core.py	Logica di business centrale: orchestrazione della pipeline AI (triage -> RAG -> risposta), raccomandazione medico.
app/extensions.py	Inizializzazione delle estensioni Flask (Flask-SQLAlchemy, Flask-Login, Flask-CORS) separata dall'app factory per evitare import circolari.
app/models.py	Modelli ORM (SQLAlchemy): definizione delle classi Paziente, Medico, Appuntamento, Messaggio con relazioni e vincoli.
app/dockerfile	Dockerfile del servizio backend: immagine Python, installazione dipendenze da requirements.txt, avvio del server Flask.
app/requirements.txt	Dipendenze Python del backend (Flask, SQLAlchemy, faiss-cpu, librerie NLP, ecc.).
faiss_index/	Indice FAISS pre-costruito con gli embeddings di MedQuAD e Asclepius. Caricato all'avvio del backend per le query RAG.
frontend/	Applicazione React: componenti UI, routing, chiamate alle API Flask.
docker-compose.yml	Orchestrazione multi-container: definisce i servizi backend (Flask), frontend (React) e database (MySQL) con le rispettive reti e volumi.
docker-entrypoint-initdb.d/	Script SQL eseguiti automaticamente da MySQL al primo avvio del container: creazione schema, tabelle e seeding dei dati iniziali.
.env / .env.example	.env contiene le variabili sensibili (escluso da .gitignore). .env.example documenta le chiavi necessarie per il setup dell'ambiente.
run.py	Entry point dell'applicazione Flask: crea l'istanza dell'app tramite l'application factory e avvia il server di sviluppo.

1.3 Architettura del Backend — Flask

Il backend è implementato con Flask, un micro-framework Python scelto per la sua leggerezza e la facilità di integrazione con librerie AI (LangChain, HuggingFace, OpenAI SDK). Segue il pattern application factory tramite app/__init__.py, con i Blueprint definiti in app/routes/ per una separazione pulita degli endpoint. Il server espone un'API REST strutturata secondo i principi CRUD e si occupa di:

- Gestire le richieste di chat e inoltrarle alla pipeline RAG (app/rag/)
- Interrogare MySQL tramite i modelli ORM definiti in app/models.py

- Eseguire il modulo di triage tramite app/core.py prima di instradare la conversazione
- Restituire al frontend risposte strutturate in JSON

1.4 Database — MySQL

MySQL gestisce l'intero strato relazionale del sistema. Le principali entità modellate sono:

- Pazienti — anagrafica, credenziali, storico medico
- Medici — specializzazione, disponibilità, slot orari
- Appuntamenti — prenotazioni con stato (attivo, cancellato, modificato)
- Messaggi chat — storico delle conversazioni per utente

In parallelo, un database vettoriale FAISS conserva gli embeddings dei dataset utilizzati dal sistema RAG: MedQuAD e Asclepius. Quest'ultimo ha sostituito MIMIC-III, originariamente previsto dalla consegna ma risultato inaccessibile: il dataset completo richiede un referente istituzionale per il download, mentre la versione demo non include il file noteevents — il file centrale per il task, contenente le note cliniche in linguaggio naturale.

1.5 Frontend — React

L'interfaccia è sviluppata in React e si articola nelle seguenti sezioni principali:

- Chat con l'assistente — finestra di dialogo con storico dei messaggi
- Risultati triage — visualizzazione del livello di urgenza rilevato
- Suggerimento medico — card con il professionista raccomandato
- Prenotazione appuntamenti — selezione slot e conferma
- Gestione prenotazioni — visualizzazione, modifica e cancellazione
- Login e registrazione — accesso con credenziali personali

2. Conformità alla Consegna

La tabella seguente riepiloga lo stato di implementazione di ciascuna funzionalità rispetto ai requisiti della challenge.

2.1 Funzionalità ad Alta Priorità

Funzionalità	Stato	Note
Chat con RAG (MedQuAD + Asclepius)	✔ **Implementato**	MIMIC-III sostituito con Asclepius per indisponibilità del file noteevents nella versione demo.
Raccomandazione medico	✔ **Implementato**	Suggerimento basato su sintomi e specializzazione
Proposta slot liberi e prenotazione	✔ **Implementato**	Visualizzazione disponibilità e booking

2.2 Funzionalità a Media Priorità

Funzionalità	Stato	Note
Visualizzazione prenotazioni	✔ Implementato	Lista prenotazioni dell'utente loggato
Storico chat	✔ Implementato	Accesso alle conversazioni precedenti tecnicamente sviluppato, ma in versione ridotta..
Upload foto diagnosi	✗ Non implementato	Non sviluppato in questo ciclo

2.3 Funzionalità a Bassa Priorità (opzionali)

Funzionalità	Stato	Note
Login / Registrazione utente	✔ Implementato	Sistema di autenticazione completo
Modifica / Cancellazione prenotazioni	✔ Implementato	CRUD completo sugli appuntamenti
Interfaccia lato medico	✗ Non implementato	Opzionale, non sviluppato
Pagina preview medico	✗ Non implementato	Opzionale, non sviluppato

3. Framework e Tecnologie Utilizzate

3.1 Flask — Backend e API REST

Flask costituisce il nucleo del backend. È stato scelto per la sua architettura minimalista, ideale per integrare librerie Python di NLP e AI senza overhead eccessivo. Nel progetto Flask viene utilizzato per:

- Esporre endpoint REST per tutte le operazioni CRUD su pazienti, medici, appuntamenti e messaggi
- Orchestrare la pipeline RAG: riceve il messaggio dell'utente, genera l'embedding, interroga il database vettoriale e compone il contesto per il modello LLM
- Eseguire il modulo di triage: classifica i sintomi riportati nella chat e assegna un livello di urgenza prima di formulare la risposta
- Gestire l'autenticazione tramite sessioni o token JWT

Durante lo sviluppo sono stati inoltre utilizzati Postman per la simulazione delle chiamate client e il testing degli endpoint REST, e XAMPP per la gestione locale del database MySQL in fase di sviluppo e debug.

3.2 MySQL — Database Relazionale

MySQL gestisce la persistenza di tutti i dati strutturati del sistema. Le tabelle principali sono progettate per supportare relazioni tra entità mediche e di scheduling. In particolare:

- La tabella medici include campi per specializzazione, disponibilità settimanale e slot prenotabili
- La tabella appuntamenti tiene traccia di stato, data, ora, paziente e medico coinvolti, con supporto a modifica e cancellazione
- La tabella chat_messages conserva ogni messaggio con riferimento all'utente e al timestamp, abilitando lo storico
- Le query SQL ottimizzate con JOIN permettono al backend di restituire in un'unica chiamata i dati aggregati necessari all'interfaccia

3.3 React — Frontend

L'interfaccia utente è costruita con React, che garantisce un'esperienza fluida grazie alla gestione dello stato e al rendering dichiarativo dei componenti. I componenti principali sviluppati sono:

- ChatWindow — gestisce l'input dell'utente, la visualizzazione dei messaggi e l'indicatore di caricamento durante le risposte AI
- TriageAlert — mostra il badge di urgenza (bassa / media / alta) restituito dal backend dopo la classificazione
- DoctorCard — visualizza il medico raccomandato con specializzazione e disponibilità
- AppointmentCalendar — permette la selezione degli slot liberi e la conferma della prenotazione
- BookingList — elenca le prenotazioni dell'utente con opzioni di modifica e cancellazione
- AuthForms — componenti di login e registrazione con validazione lato client

4. Sistema RAG

4.1 Modello di embedding e scelta del modello

Il sistema RAG utilizza `sentence-transformers/all-MiniLM-L6-v2` come modello di embedding, configurato con `normalize_embeddings=True` per garantire che le distanze coseno siano calcolate correttamente. La scelta è ricaduta su questo modello per il buon compromesso tra velocità di inferenza e qualità delle rappresentazioni semantiche. In fase di sviluppo è stata valutata come alternativa `BAAI/bge-base-en-v1.5`, modello più accurato ma significativamente più lento, che rimane disponibile nel codice come opzione commentata per eventuali deploy con risorse computazionali maggiori.

4.2 Logica di fusion

I risultati delle ricerche sui due indici (MedQuAD e Asclepius) vengono uniti e deduplicati tramite un metodo di fusion con score. La deduplicazione avviene confrontando i primi 120 caratteri del contenuto di ciascun documento, in modo da eliminare chunk sostanzialmente identici presenti in entrambi i dataset. L'elenco risultante viene ordinato per distanza coseno (cosine distance) crescente, così che i chunk più semanticamente vicini alla query si trovino sempre in cima al contesto passato al modello.

La distanza FAISS viene convertita in un valore di confidence nell'intervallo [0-1]. Questo valore viene incluso nel JSON di risposta al frontend, dove può essere utilizzato per indicare

all'utente il grado di affidabilità della risposta rispetto alle fonti disponibili. Un confidence basso segnala che nessun chunk sufficientemente rilevante è stato trovato nei dataset.

4.3 Classificazione semantica della query

Prima di avviare il retrieval completo, il sistema esegue una classificazione semantica. La funzione recupera i 4 chunk più vicini alla query e valuta la distanza del migliore: score inferiore a 0,4 classifica la query come medica, tra 0,4 e 0,7 come incerta, oltre 0,7 come non medica. Questa soglia permette di identificare rapidamente domande fuori dominio. Il filtro che avrebbe bloccato le richieste non mediche è stato deliberatamente disabilitato nel codice, poiché il bot risultava eccessivamente restrittivo anche su richieste legittime formulate in modo indiretto, rappresentando un trade-off consapevole tra sicurezza del dominio e usabilità.

4.4 Gestione della memoria conversazionale

La memoria della conversazione è gestita tramite un riassunto degli ultimi 12 messaggi, generato prima di ogni risposta. Il messaggio dell'utente viene salvato in memoria prima della generazione del riassunto, in modo che il contesto più recente sia sempre incluso. Il riassunto viene costruito tramite prompt specifico che chiede al modello di estrarre solo le informazioni clinicamente rilevanti, evitando di inflazionare il contesto con parti di conversazione irrilevanti per la risposta medica.

5. AI Feature Aggiuntiva: Sistema di Triage

Il requisito obbligatorio di integrare almeno un'ulteriore funzionalità di intelligenza artificiale è stato soddisfatto tramite un modulo di triage automatico dei sintomi. Questo modulo rappresenta una delle aggiunte più rilevanti e differenzianti dell'intero progetto.

5.1 Obiettivo

Il triage medico è il processo con cui i pazienti vengono classificati in base all'urgenza della loro condizione, al fine di prioritizzare le cure e indirizzarli verso il professionista più adatto. Nel contesto dell’assistente virtuale, il sistema include un modulo di triage medico basato sull'intelligenza artificiale che valuta l'urgenza dei sintomi e individua potenziali segnali di allarme, migliorando la sicurezza dei pazienti e l’esperienza dell’utente con il chatbot.

5.2 Funzionamento

Il flusso del modulo di triage si articola come segue:

- L'utente descrive i propri sintomi tramite la chat in linguaggio naturale
- Il backend estrae le entità cliniche rilevanti dal testo (sintomi, durata, intensità) tramite NLP
- Un classificatore assegna un livello di urgenza tra tre categorie: Bassa, Media, Alta
- Il risultato viene restituito al frontend come badge visivo colorato e come input per la raccomandazione del medico
- In caso di urgenza alta, il sistema mostra un avviso esplicito che invita a contattare il pronto soccorso

5.3 Tecnologia

Il classificatore è implementato tramite un modello di NLP fine-tunato su dataset clinici, integrato via Flask. La classificazione sfrutta i pattern lessicali del testo per associare sintomi a categorie di urgenza, con un approccio basato su embedding semantici che garantisce robustezza anche a frasi non standardizzate: un componente di riformulazione delle query ottimizza il recupero RAG traducendo il linguaggio naturale in query rilevanti dal punto di vista medico.

Entrando più nel dettaglio, Il modulo di triage è integrato direttamente nella pipeline principale che coordina le risposte all'utente. Quando l'utente invia un messaggio, il sistema esegue in parallelo un filtro semantico, che interroga l'indice FAISS per determinare se la query è medica, incerta o non medica, e delega a un LLM l'estrazione strutturata di intent, sintomi, severity e livello di triage in un'unica invocazione. Questo approccio elimina la necessità di una chiamata separata per la classificazione dell'intent, riducendo la latenza complessiva.

Nel caso di triage classificato come 'urgente', la pipeline bypassa completamente il sistema RAG e restituisce immediatamente una risposta di emergenza, senza attendere il retrieval. In tutti gli altri casi, il flusso prosegue con il recupero del contesto da FAISS, che fonde i risultati di MedQuAD e Asclepius ordinando i chunk per *cosine distance*, e la generazione della risposta tramite LLM. Il JSON restituito al frontend include sempre severit, triage e confidence.

5.4 Valore Aggiunto

Il modulo di triage differenzia il progetto rispetto a un semplice chatbot medico, avvicinandolo a un sistema di supporto clinico realistico. Permette di:

- Ridurre il rischio che sintomi gravi vengano gestiti superficialmente
- Ottimizzare il matching tra paziente e medico specialista
- Fornire all'utente un feedback immediato sulla gravità percepita della propria condizione

6. Punti Forti e Aree di Miglioramento

6.1 Punti Forti

Copertura funzionale solida

Il progetto copre tutte e tre le funzionalità ad alta priorità della consegna (chat RAG, raccomandazione medico, gestione appuntamenti) e aggiunge due funzionalità opzionali (login e modifica/cancellazione prenotazioni), dimostrando un livello di completamento superiore alla baseline richiesta.

Architettura coerente e scalabile

La separazione netta tra frontend React, backend Flask e database MySQL segue principi architetturali consolidati. Questa struttura facilita la manutenzione, il testing e l'estensione futura del sistema.

Sistema RAG su dataset specializzati

L'utilizzo combinato di MedQuAD (domande e risposte mediche) e Asclepius (dataset clinico in linguaggio naturale, adottato in sostituzione di MIMIC-III) per alimentare il sistema RAG

garantisce risposte contestuali di qualità superiore rispetto a modelli generici, con un'aderenza al dominio sanitario particolarmente rilevante. La scelta di Asclepius è motivata da ragioni concrete: MIMIC-III richiedeva un referente istituzionale per l'accesso, e la versione demo era priva del file *noteevents*, il più rilevante per il task di comprensione clinica.

Triage come differenziatore clinico

Il modulo di classificazione dei sintomi aggiunge un valore concreto e clinicamente rilevante al sistema, rendendolo più sicuro e utile rispetto a un semplice Q&A medico. È la funzionalità più innovativa dell'intero progetto.

Gestione appuntamenti CRUD completa

La possibilità di creare, visualizzare, modificare e cancellare prenotazioni — con gestione degli slot liberi — rappresenta un caso d'uso end-to-end completamente funzionante.

6.2 Aree di Miglioramento

Upload di immagini diagnostiche

La funzionalità di caricamento foto (analisi, referti) non è stata implementata. Si tratta di un'estensione rilevante, che potrebbe essere realizzata integrando un modello multimodale (es. LLaVA o GPT-4V) in grado di analizzare immagini mediche e inserire l'output nel contesto RAG.

Interfaccia lato medico

Manca un pannello dedicato ai professionisti sanitari. Una dashboard medico permetterebbe di visualizzare le prenotazioni in entrata, consultare la chat del paziente prima della visita e gestire la propria disponibilità, rendendo il sistema completo anche dal lato dell'offerta.

Accuratezza e validazione del triage

Il modulo di triage, pur funzionante, non ha ancora una validazione clinica formale. In un contesto reale, i livelli di urgenza dovrebbero essere calibrati su protocolli medici standardizzati (es. Manchester Triage System) e validati da professionisti del settore.

Gestione della sessione e sicurezza

Il sistema di autenticazione potrebbe essere rafforzato con meccanismi come il refresh token, la gestione della scadenza della sessione, e la cifratura end-to-end delle chat, requisiti fondamentali in ambito GDPR e sanitario.

Qualità del RAG e chunking

La pipeline RAG potrebbe beneficiare di strategie più avanzate di chunking e re-ranking dei documenti recuperati, per ridurre le allucinazioni e migliorare la pertinenza delle risposte, specialmente su query complesse con termini clinici ambigui. Inoltre, è presente un lag all’avvio della chat col primo messaggio a causa del load dei dati che, per quanto non dannoso, potrebbe essere eliminato.

Bug vari

Il progetto presenta alcune funzionalità tecnicamente sviluppate, ma non perfettamente funzionanti o che potrebbero essere testate più estensivamente. Avendo la possibilità di dilungarsi sul progetto per curarne questi aspetti, sono in realtà bug di semplice risoluzione.

7. Conclusione

Il progetto Healthcare Assistant dimostra un'implementazione concreta e funzionale di un assistente virtuale per il settore sanitario. Le scelte tecnologiche adottate (Flask, MySQL, React) sono coerenti con i requisiti della consegna e con le esigenze di un sistema AI-powered in ambito medico.

Il sistema RAG su dataset specializzati e il modulo di triage rappresentano i punti di maggiore valore tecnico e clinico del progetto. Le aree di miglioramento identificate, in particolare l'interfaccia medico, l'analisi di immagini diagnostiche e la validazione del triage, costituiscono una direzione chiara per possibili evoluzioni future.

Nel complesso, il progetto soddisfa pienamente i requisiti obbligatori e le priorità alte della challenge, con uno sforzo aggiuntivo sulle funzionalità opzionali.